Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

ŞL.dr.ing. Andrei Mogoş

2020-2021

- Complexitatea algoritmilor (1) -

Generalităţi

Observaţie: Teoria complexităţii algoritmilor se ocupă de problemele rezolvabile mecanic.

Definiţie: **Complexitatea unui algoritm** se referă la cantitatea de resurse de calcul (timp şi spaţiu de memorie) necesare execuţiei algoritmului.

Observaţie: Spunem că un algoritm are **complexitate temporală** (respectiv **complexitate spaţială**) cu atât mai mare cu cât timpul (respectiv spaţiul de memorie) cerut de execuţia algoritmului este mai mare.

Dimensiunea unei probleme (1)

Definiţie: Dimensiunea unei probleme este o măsură compusă a acelor date care influenţează major performanţele algoritmului. Dimensiunea problemei se măsoară ca lungime a şirului de simboluri care reprezintă datele respective, simbolurile fiind considerate atomice din punct de vedere al prelucrărilor efectuate de algoritm.

Observaţie: Modul în care calculăm dimensiunea problemei depinde de tipul structurii de date şi de identificarea simbolurilor atomice.

Dimensiunea unei probleme (2)

Exemplul 1: Pentru algoritmii care prelucrează structuri de date reprezentate prin simboluri atomice, dimensiunea problemei este numărul de elemente din structură.

P1: Se consideră un vector cu *n* elemente numere reale. Să se sorteze crescător acest vector.

dim(P1) = *n*

Din punct de vedere al sortării elementele vectorului sunt simboluri atomice.

Dimensiunea unei probleme (3)

Exemplul 2: Pentru algoritmii pentru grafuri G=(V,E) dimensiunea problemei este lungimea unui şir de simboluri care reprezintă nodurile din V şi arcele din E ale grafului.

Dacă graful are *n* noduri şi *m* arce atunci dimensiunea problemei depinde de *n* şi *m*.

P2: nodurile: 1, 2, 3, …, arcele: (1, 2), (1, 3), ...

dim(P2) = *n*+*2m*** (de obicei se foloseşte ***n*+*m***)

P3: nodurile: 1, 2 ,3, …, arcele: 1, 2, 3, …

dim(P3) = *n*+*m*

Dimensiunea unei probleme (4)

Exemplul 3: Pentru algoritmii numerici trebuie să se ţină cont de diferenţa dintre valoarea datelor prelucrate şi dimensiunea reprezentării lor.

P4: Fie *n* >= 2 un număr natural. Să se decidă dacă *n* este număr prim.

```
prim(n)
    for d = 2 … ⌊√n⌋
        if (n mod d = 0) return 0
    return 1
```

C(prim) = $O(\sqrt{n})$ **dim(P4) = n? NU**

Dimensiunea unei probleme (5)

0 0 0 0 | 0
0 0 0 1 | 1
0 0 1 0 | 2
0 0 1 1 | 3
0 1 0 0 | 4
0 1 0 1 | 5
0 1 1 0 | 6
0 1 1 1 | 7
1 0 0 0 | 8
.
.
.

k = numărul de biţi din reprezentarea lui *n.*

$$2^{k-1} \le n < 2^k$$

$$k - 1 \le \log_2 n < k$$

$$k - 1 = \lfloor \log_2 n \rfloor$$

$$k = \lfloor \log_2 n \rfloor + 1 \approx \log_2 n$$

$$n \approx 2^k \Rightarrow \sqrt{n} \approx 2^{0.5*k}$$

C(prim) = $O(\sqrt{n}) = O(2^{0.5*k})$

dim(P4) = *k*

Ce complexitate măsurăm?

- în cazul cel mai favorabil
- în cazul cel mai defavorabil
- în cazul mediu
- complexitate amortizată

Cum măsurăm complexitatea?

- **Exact** (metrică exactă, cantitativ)
  - **Toate operaţiile** (complexitate exactă)
  - **Operaţiile critice** (o aproximare a complexităţii exacte)
- **Aproximativ** (metrică aproximativă, calitativ)
  - **Clase de complexitate**

Măsurarea exactă a complexităţii (1)

Definiţie: Metrica (exactă) stabileste cantitatea de resurse folosite de operaţiile elementare ale unui algoritm.

Definiţie: **Metrica unitară şi omogenă**: fiecare *operaţie elementară* a unui algoritm cere o unitate convenţională de timp (spaţiu).

Definiţie: Metrica neomogenă: fiecare *operaţie* a unui algoritm foloseşte o anumită cantitate de resurse măsurată folosind unităţi convenţionale de timp (spaţiu).

Măsurarea exactă a complexităţii (2)

Exemplu: Considerăm instrucţiunea **v[i] = a + 5.**

- Costul folosind metrica unitară şi omogenă: **3**

(accesarea valorii prin index, suma şi atribuirea)

- Costul folosind metrica neomogenă: **c**

(unde c este o constantă pozitivă (număr real))

Observaţie: **Operaţiile elementare** sunt operaţiile foarte simple care se presupune că se execută într-un timp foarte scurt (instantaneu): accesarea unei valori prin indecsi, operaţii aritmetice, operaţii logice, comparaţii, etc.

Măsurarea exactă a complexităţii (3)

Definiţie: Operaţiile critice sunt operaţiile executate cel mai frecvent sau cele mai complicate din punct de vedere al complexităţii.

Exemple:

op1 (O(n))	op1 (O(n))	op1 (O(n^2))
for i = 1 .. n	for i = 1 .. n	for i = 1 .. n
op2 (O(1))	op2 (O(n))	op2 (O(1))
operaţii critice:	operaţii critice:	operaţii critice:
op1, op2	op2	op1

Măsurarea exactă a complexităţii (4)

Definiţie: Complexitatea exactă a unui algoritm este caracterizată de o funcţie $f : N^* \to R_+^*$ numită *funcţie de complexitate*. Valoarea $f(n)$ reprezintă cantitatea de resurse consumate de algoritm pentru dimensiunea *n* a problemei.

a) Măsurarea complexităţii utilizând **toate operaţiile** reprezintă *aflarea funcţiei de complexitate* f .

b) Măsurarea complexităţii utilizând **operaţiile critice** reprezintă *aflarea unei aproximări a funcţiei de complexitate* f .

Exemplu: Sortarea prin inserţie (1)

Problema:

Să se sorteze crescător vectorul *v* cu *n* elemente numere reale folosind **sortarea prin inserţie**.

Convenţii:

- presupunem că pentru instrucţiunile **for** şi **while** antetul se execută o dată în plus faţă de corpul instrucţiunii.

- comentariile nu se iau în considerare la calculul complexităţii.

Analizăm complexitatea temporală a algoritmului construit folosind metrica neomogenă.

Exemplu: Sortarea prin inserţie (2)

ins(v)	cost	repetiţii
for i = 2 .. n	c_1	n
aux = v[i]	c_2	$n-1$
j = i – 1	c_3	$n-1$
while j >= 1 and v[j] > aux	c_4	$\sum_{i=2}^{n} t_i$
v[j + 1] = v[j]	c_5	$\sum_{i=2}^{n} (t_i - 1)$
j = j – 1	c_6	$\sum_{i=2}^{n} (t_i - 1)$
v[j + 1] = aux	c_7	$n-1$

Exemplu: Sortarea prin inserţie (3)

a) Toate operaţiile:

$$T(n) = c_1 \cdot n + c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot \sum_{i=2}^{n} t_i +$$

$$+ c_5 \cdot \sum_{i=2}^{n} (t_i - 1) + c_6 \cdot \sum_{i=2}^{n} (t_i - 1) + c_7 \cdot (n-1)$$

a1) Cazul cel mai favorabil ($t_i = 1$):

$$T(n) = c_1 \cdot n + c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot (n-1) + c_7 \cdot (n-1) =$$

$$= (c_1 + c_2 + c_3 + c_4 + c_7) \cdot n - (c_2 + c_3 + c_4 + c_7) = \Theta(n)$$

Exemplu: Sortarea prin inserţie (4)

a2) Cazul cel mai defavorabil ($t_i = i$):

$$T(n) = c_1 \cdot n + c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot \sum_{i=2}^{n} i +$$

$$+ c_5 \cdot \sum_{i=2}^{n} (i-1) + c_6 \cdot \sum_{i=2}^{n} (i-1) + c_7 \cdot (n-1) = \ldots =$$

$$= (\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2}) \cdot n^2 + (c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7) \cdot n -$$

$$- (c_2 + c_3 + c_4 + c_7) = \Theta(n^2)$$

Exemplu: Sortarea prin inserţie (5)

a3) Cazul mediu ($t_{i-1} = \frac{0+(i-1)}{2} \Rightarrow t_i = \frac{i+1}{2}$ **):**

$$T(n) = c_1 \cdot n + c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot \sum_{i=2}^{n} \frac{i+1}{2} +$$

$$+ c_5 \cdot \sum_{i=2}^{n} \frac{i-1}{2} + c_6 \cdot \sum_{i=2}^{n} \frac{i-1}{2} + c_7 \cdot (n-1) = \ldots =$$

$$= \left(\frac{c_4}{4} + \frac{c_5}{4} + \frac{c_6}{4}\right) \cdot n^2 + \left(c_1 + c_2 + c_3 + \frac{3 \cdot c_4}{4} - \frac{c_5}{4} - \frac{c_6}{4} + c_7\right) \cdot n -$$

$$- (c_2 + c_3 + c_4 + c_7) = \Theta(n^2)$$

Exemplu: Sortarea prin inserţie (6)

b) Operaţii critice (fiecare caz se tratează separat):

b1) Cazul cel mai favorabil, operaţii critice: 2, 3, 4, 7 ($t_i = 1$):

$$T(n) = c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot \sum_{i=2}^{n} 1 + c_7 \cdot (n-1) =$$

$$= \ldots = (c_2 + c_3 + c_4 + c_7) \cdot n - (c_2 + c_3 + c_4 + c_7) = \Theta(n)$$

Exemplu: Sortarea prin inserţie (7)

b2) Cazul cel mai defavorabil, operaţii critice: 5, 6 ($t_i = i$):

$$T(n) = c_5 \cdot \sum_{i=2}^{n} (i-1) + c_6 \cdot \sum_{i=2}^{n} (i-1) = \ldots =$$

$$= (\frac{c_5}{2} + \frac{c_6}{2}) \cdot n^2 - (\frac{c_5}{2} + \frac{c_6}{2}) \cdot n = \Theta(n^2)$$

Exemplu: Sortarea prin inserţie (8)

b3) Cazul mediu, operaţii critice: 5, 6

($t_i = \frac{i+1}{2}$ **):**

$$T(n) = c_5 \cdot \sum_{i=2}^{n} \frac{i-1}{2} + c_6 \cdot \sum_{i=2}^{n} \frac{i-1}{2} = \ldots =$$

$$= (\frac{c_5}{4} + \frac{c_6}{4}) \cdot n^2 - (\frac{c_5}{4} + \frac{c_6}{4}) \cdot n = \Theta(n^2)$$

Exemplu: Sortarea prin inserţie (9)

	Toate operaţiile	Operaţii critice
Cazul cel mai favorabil	$\Theta(n)$	$\Theta(n)$
Cazul cel mai defavorabil	$\Theta(n^2)$	$\Theta(n^2)$
Cazul mediu	$\Theta(n^2)$	$\Theta(n^2)$